

Query Processing of Simple Relational Algebra Expressions

Uni Auckland Databases 51

Gerald Weber

Generality of θ -Join (Theta-Join)

- ▶ The θ -join ($\bowtie_{\theta}$) is a general form of join in relational algebra.
- ▶ It generalizes several other join operations:
 - ▶ **Equi-Join**: A join where θ consists of only equality conditions.
 - ▶ **Range Join**: A join where θ involves a range of values.
 - ▶ **Theta-Join**: The most general form, where θ can be any condition.
- ▶ Example:

$$R \bowtie_{\theta} S \quad \text{where} \quad \theta = (R.A < S.A) \vee (R.B > S.B)$$

- ▶ The θ -join allows for complex conditions, making it a powerful tool for query processing.

Introduction

- ▶ Focus on a single θ -join with selections (σ) and projections (π).
- ▶ Question of pushing projections (π) and selections (σ) outward (to the root of the expression tree).
- ▶ Example formula:

$$\pi_A(\pi_C(\sigma_{\theta_1}(R)) \bowtie_{\theta} \pi_E(\sigma_{\theta_2}(S)))$$

- ▶ We might also use a natural join (projection implied)

Pushing Selections Outward

- ▶ assume a condition (Z) involves only attributes of one relation (R).

- ▶ Example:

$$\sigma_Z(R \bowtie S) = (\sigma_Z(R)) \bowtie S$$

- ▶ Cases to consider:

- ▶ Selections can reduce the size of intermediate results.
- ▶ Selections can be pushed outward to avoid intermediate results.

Pushing Projections Outward

- ▶ Projections can be pushed outward assuming no name conflicts:
- ▶ Example:

$$\pi_{A,B}(\underline{R} \bowtie \underline{S}) = (\pi_{\underline{A}}(R)) \bowtie (\pi_{\underline{B}}(S))$$

no common attribute.

条件不能丢

- ▶ Cases to consider:
 - ▶ Projections can be pushed inward to reduce the number of columns in intermediate results.
 - ▶ Projections can be pushed outward to avoid intermediate results.

Types of Selection

▶ Tuplewise Selection:

- ▶ Filters tuples based on specific attribute values or conditions.
- ▶ Must be computed for every tuple.
- ▶ Example: Selecting tuples fulfilling a complex condition:

$$\sigma_{(C=3*C) \vee (B>2*B)}(R)$$

▶ Range Selection:

- ▶ Filters tuples based on a range of attribute values.
- ▶ Efficient when tuples are accessible by range (i.e., sorted).
- ▶ Example:

$$\sigma_{5 \leq A}(R)$$

- ▶ Selects tuples where attribute A is larger than 5.

- ▶ Example:

$$\sigma_{5 \leq A \leq 10}(R)$$

- ▶ Selects tuples where attribute A is between 5 and 10.

直接访问
sorted data
index

Equality Selection

► Equality as **Range**:

- Equality condition can be expressed as a range.
- Example:

$$\sigma_{A=b}(R) \equiv \sigma_{A \leq b \wedge A \geq b}(R)$$

- Selects tuples where attribute A equals b .

► Equality with **Hashing**:

- Equality condition can be efficiently implemented using hashing.
- Hashing allows for quick lookup of tuples with specific attribute values.
- Example:

$$\sigma_{A=b}(R) \text{ using a hash index on } A$$

- Selects tuples where attribute A equals b using a hash index.

5/13/14

Example

► **Query:** Find the names of all items with value over \$100 from order with timestamp '202504011501'.

► **Relations:**

► **Orders**(OrderID, CustomerID, Time)

► **OrderItems**(OrderID, ItemID, ItemName, Value)

► **Query Proposal:**

$\pi_{ItemName}(\sigma_{Time='202504011501'}(Orders \bowtie \sigma_{Value > 100}(OrderItems)))$

► **Is this the best we can do?**

Example

- ▶ **Query:** Find the names of all items with value over \$100 from order with timestamp '202504011501'.

- ▶ **Relations:**

- ▶ **Orders**(OrderID, CustomerID, Time)
- ▶ **OrderItems**(OrderID, ItemID, ItemName, Value)

- ▶ **Query Proposal:**

$$\pi_{ItemName}(\sigma_{Time='202504011501'}(Orders \bowtie \sigma_{Value>100}(OrderItems)))$$

- ▶ **Is this the best we can do?**

- ▶ Is the selection on Value > 100 applied too early? ✓
- ▶ Is the selection on Time = '202504011501' applied too late? ✓

Example Data for Natural Join

OrderID	CustomerID	Time	ItemID	ItemName	Value
1001	C001	2025-04-01 15:01	I001	ItemA	150
1001	C001	2025-04-01 15:01	I002	ItemB	90
1001	C001	2025-04-01 15:01	I006	ItemF	110
1002	C002	2025-04-01 15:02	I003	ItemC	200
1002	C002	2025-04-01 15:02	I007	ItemG	130
1003	C003	2025-04-01 15:03	I004	ItemD	120
1003	C003	2025-04-01 15:03	I005	ItemE	80
1003	C003	2025-04-01 15:03	I008	ItemH	140
1004	C004	2025-04-01 15:04	I009	ItemI	95
1004	C004	2025-04-01 15:04	I010	ItemJ	105

Example: Pushing Selections and Projections

► Query Proposal:

$$\pi_{ItemName}(\sigma_{Time='202504011501'}(Orders \bowtie \sigma_{Value>100}(OrderItems)))$$

► Optimized Query:

$$\pi_{ItemName}(\sigma_{Value>100}(\sigma_{Time='202504011501'}(Orders) \bowtie OrderItems))$$

► Why This Works:

- Selection on Time = '202504011501' is pushed to the Orders table.
- The selection on ItemValue > 100 is applied after the join; no intermediate table needed.

How the Join Works

- ▶ **Assumption:** The OrderItems table is clustered by OrderID or has an index on OrderID.
- ▶ **Clustering:**
 - ▶ Clustering means that the rows in the OrderItems table are physically stored in the order of OrderID.
 - ▶ This allows for efficient retrieval of all items for a specific order.
- ▶ **Join Process:**
 - ▶ **Step 1:** Apply the selection on Orders to find OrderID = 1002.
 - ▶ **Step 2:** Use the index or clustering on OrderItems to quickly locate all items for OrderID = 1002.
 - ▶ **Step 3:** Perform the join between the filtered Orders and OrderItems.
- ▶ **Benefits:**
 - ▶ Efficiently narrows down the relevant tuples using the index or clustering.
 - ▶ Reduces the size of intermediate results, improving query performance.

Conclusion

- ▶ Selections and projections can be repositioned around a join to improve query processing.
- ▶ Careful consideration must be given if we want intermediate results.
- ▶ Sorting/Clustering is important.